Residual Frontier Refinement: Exact Shortest-Path Propagation with Adaptive Partial Ordering

Morgan Petrik
Militant.AI
mpetrik@militant.ai

DOI (v1.0.0): 10.5281/zenodo.20329104

22 May 2026

Abstract

Shortest-path solvers based on global priority queues impose a total order on the active frontier. This ordering is sufficient for exact propagation, but it is often stronger than necessary on structured graph and spatial routing problems. Residual Frontier Refinement (RFR) is an exact single-source shortest-path method that replaces global heap ordering with adaptive partial ordering over distance bands. A band is processed as a batch only when conservative safety checks prove that no unprocessed candidate can invalidate the propagation order. Ambiguous bands are split or resolved by a local exact fallback.

This whitepaper formalises the RFR algorithm, describes its safety invariant, defines its work model, and reports empirical observations from generated graph families and topographic routing demos. The current implementation validates exactness against DIJKSTRA across tested graph families. It also demonstrates that structured frontiers can avoid global ordering operations and expose useful diagnostic information about frontier ambiguity. The current Python implementation does not yet validate universal wall-clock superiority over DIJKSTRA on pre-built graphs; its strongest validated claim is that shortest-path propagation can preserve exactness while replacing full global ordering with conservative residual frontier resolution.

1 Contributions

This whitepaper makes four contributions:

1. It defines Residual Frontier Refinement as an exact shortest-path propagation method based on adaptive partial ordering.
2. It gives the conservative safety condition used to decide when a frontier band may be batch-processed.
3. It separates global heap-ordering work from local residual-resolution work, giving a clearer account of what the algorithm changes.
4. It reports the present empirical and operational status: exactness is validated across tested graph families, work-shape benefits are visible, and universal wall-clock superiority is not yet established for the Python implementation.

2 Problem Statement

Let $G = (V, E, w)$ be a directed graph with non-negative edge weights $w(u, v) \geq 0$, source $s \in V$, and shortest-path distances

$$d^\star(v) = \min_{p: s \rightsquigarrow v} \sum_{(u,z) \in p} w(u, z).$$

The single-source shortest-path problem computes $d^\star(v)$ for all reachable vertices and may also record predecessors for path reconstruction.

DIJKSTRA's algorithm solves this problem by repeatedly extracting the globally minimum unsettled frontier element. The global minimum rule is exact, but it enforces a total order over the frontier. On structured graphs, many frontier vertices may be provably safe to process together, or safe to resolve using only local ordering, because their relative order cannot change the final distances. The core RFR hypothesis is:

> Shortest-path propagation requires enough ordering to preserve correctness, not necessarily complete global ordering at every step.

RFR tests this hypothesis by replacing the global heap with distance bands, conservative residual checks, splitting, and local fallback.

3 Algorithm Overview

3.1 Residual Frontier

RFR maintains a frontier $\mathcal{F}$ partitioned into bands. Each band $\mathcal{B}$ has:

- a lower and upper distance bound $[\ell(\mathcal{B}), u(\mathcal{B}))$,
- a set of active members $\{(v, \hat{d}(v))\}$,
- update counts and refinement depth,
- residual diagnostics measuring internal width, volatility, cross-edge risk, and degree risk.

The implementation records the following residual components for a band:

$$R(\mathcal{B}) = R_{\text{width}}(\mathcal{B}) + R_{\text{volatility}}(\mathcal{B}) + R_{\text{cross}}(\mathcal{B}) + R_{\text{degree}}(\mathcal{B}).$$

These residuals are not trusted as correctness criteria by themselves. They guide splitting and local fallback. Exactness depends on the conservative safety rules in Section 3.3.

3.2 High-Level Procedure

```
ResidualFrontierSSSP(G, s):
    dist[v] <- infinity for all v
    dist[s] <- 0
    frontier.insert_or_update(s, 0)

    while frontier is not empty:
        B <- lowest non-empty distance band
```

```
if Safe(B, frontier, dist, G):
    batch <- all members of B sorted locally by tentative distance
    remove B from frontier
else:
    residual <- measure residual pressure in B
    if residual exceeds threshold and B can be refined:
        split B into smaller bands
        continue
    else:
        batch <- one local exact minimum from B

for (u, du) in batch:
    if u is stale or settled:
        continue
    settle u
    for each outgoing edge (u, v, w):
        candidate <- du + w
        if candidate < dist[v]:
            dist[v] <- candidate
            predecessor[v] <- u
            if v is unsettled:
                frontier.insert_or_update(v, candidate)
```

3.3 Safe Batch Extraction

Let $\mathcal{B}$ be the lowest non-empty band in the frontier. Let

$$d_{\max}(\mathcal{B}) = \max_{v \in \mathcal{B}} \hat{d}(v)$$

and let

$$d_{\min}(\mathcal{F} \setminus \mathcal{B}) = \min_{v \in \mathcal{F} \setminus \mathcal{B}} \hat{d}(v),$$

with $d_{\min} = \infty$ if no outside frontier elements exist.

RFR first requires:

$$d_{\max}(\mathcal{B}) \leq d_{\min}(\mathcal{F} \setminus \mathcal{B}).$$

This ensures no already-known outside frontier element precedes the band.

The implementation also checks whether relaxing a member of $\mathcal{B}$ could create an outside candidate that should precede remaining band members. For each $u \in \mathcal{B}$ and outgoing edge (u, v) with $v \notin \mathcal{B}$, the safety check rejects the batch if:

$$\hat{d}(u) + w(u, v) < d_{\max}(\mathcal{B}).$$

If this condition occurs, processing the whole band could skip an outside candidate that should be interleaved before later band members. The band is then unsafe and must be split or resolved locally.

3.4 Unsafe Bands

Unsafe bands represent frontier ambiguity. RFR responds in two ways:

1. **Split:** If the band has multiple members and has not exceeded the maximum refinement depth, it is split into narrower bands.
2. **Local exact fallback:** If splitting is no longer useful or allowed, the algorithm extracts the exact local minimum from the band.

The local fallback is not a correctness failure. It is a controlled reversion to exact ordering where partial ordering is insufficient.

3.5 Compositional Design

RFR is intentionally decomposed into small, independently inspectable parts:

- **Safety predicates** decide whether a frontier band can be processed without violating exactness.
- **Band transformations** insert, update, split, and remove frontier regions without settling vertices.
- **Residual measurements** describe ambiguity pressure but do not replace the safety predicates.
- **Local fallback** provides controlled recovery to exact local ordering when partial ordering is insufficient.

This separation is operationally important. A conventional global priority queue hides most ordering pressure inside heap operations. RFR exposes that pressure as data: a band was safe, unsafe, split, locally resolved, or accepted as a batch. The result is not only a distance field, but also a diagnostic trace of where the problem required precision.

4 Correctness Argument

4.1 Invariant

RFR maintains the standard label-setting invariant:

> When a vertex u is settled, its tentative distance $\hat{d}(u)$ equals the true shortest-path distance $d^\star(u)$.

Dijkstra enforces this invariant by extracting the global minimum unsettled label. RFR enforces it by proving that either a full band is safe, or by falling back to an exact local minimum within an unsafe band.

4.2 Safe Band Lemma

Lemma. If a band $\mathcal{B}$ passes the two safety checks in Section 3.3, then settling the members of $\mathcal{B}$ in nondecreasing tentative-distance order preserves the label-setting invariant.

Sketch. The first safety check ensures that no known outside frontier vertex has a smaller tentative distance than any member of $\mathcal{B}$. The second safety check ensures that relaxing any band member cannot create an outside candidate whose distance is smaller than the maximum remaining distance within the band. Therefore, no outside candidate needs to be interleaved before the batch completes. Sorting the batch internally by tentative distance gives the same order that a global priority queue would observe over this locally isolated subset. Thus each settled vertex has the globally minimal necessary label at its settlement point.

4.3 Unsafe Band Lemma

Lemma. If a band is unsafe, splitting or extracting a local exact minimum preserves the possibility of exact label-setting propagation.

Sketch. Splitting does not settle any vertex; it only refines the frontier partition. It therefore cannot invalidate distances. Local exact fallback extracts the minimum label inside the current lowest band. Since the band is the lowest non-empty band by lower bound, and exact local extraction is used when batch safety is unavailable, this step conservatively approximates DIJKSTRA's global-minimum behaviour within the ambiguous region. Stale entries are ignored, and relaxations are accepted only when they improve labels.

4.4 Theorem

Theorem. For graphs with non-negative edge weights, RFR computes the same shortest-path distances as DIJKSTRA, assuming the safety checks and stale-entry rules described above.

Sketch. By induction over settled vertices. The source is settled with distance zero. At each step, either a safe band is processed under the Safe Band Lemma or an unsafe band is refined/resolved under the Unsafe Band Lemma. Relaxation is identical to DIJKSTRA once a vertex is selected. Since no settled vertex violates the label-setting invariant, all reachable final distances match $d^\star$. Unreachable vertices remain at infinity.

5 Work Model

RFR separates two forms of work that are conflated in ordinary priority-queue shortest-path implementations.

5.1 Dijkstra Work

The baseline global ordering work is:

$$W_{\text{global}} = H_{\text{push}} + H_{\text{pop}},$$

where H_{push} and H_{pop} are heap insertions and heap removals. The implementation records these as `global_heap_pushes` and `global_heap_pops`.

5.2 RFR Work

RFR records local resolution work:

$$W_{\text{local}} = S_{\text{band}} + R_{\text{local}} + F_{\text{heap}} + I_{\text{heap}},$$

where S_{band} is band scan work, R_{local} is local refinement count, F_{heap} is local heap fallback count, and I_{heap} is the number of items resolved by local fallback.

The implementation also records safe batches, band splits, frontier peak size, stale entries, relaxations, distance updates, and batch vertices.

5.3 Resolution Efficiency

The project uses a diagnostic ratio:

$$E = \frac{W_{\text{global avoided}}}{W_{\text{local}}}.$$

Values above one indicate that RFR avoided more global heap-ordering operations than it added in local resolution work. This is a work-shape metric, not a direct wall-clock claim.

6 Adaptive Policies

6.1 Static Graph Classification

The implementation computes graph features:

- vertex and edge counts,
- density,
- average degree,
- clustering,
- edge-weight mean and variance.

These features select an initial frontier policy:

Graph signal	Policy response
Road-like geometry	broad directional bands, larger scan limit
Cluster structure	basin-first propagation, moderate bands
High ambiguity	narrow bands and local heap fallback
Low ambiguity	broad bands with moderate residual threshold
Medium ambiguity	split-oriented hybrid bands

6.2 Probe Revision

RFR v4 introduced a probe phase:

$$\text{graph features} \rightarrow \text{initial policy} \rightarrow \text{frontier probe} \rightarrow \text{revised policy}.$$

The probe observes split pressure, fallback rate, safe-batch ratio, residual size, cross-edge risk, and volatility. RFR v5 removes duplicate traversal by making the probe the opening segment of the exact run; the policy is revised in place and propagation continues.

6.3 Hybrid Indexed Frontier

RFR v6 adds a hybrid frontier:

$$\begin{cases} \text{indexed residual frontier}, & \text{structured policies}, \\ \text{split-capable v5 frontier}, & \text{ambiguous policies}. \end{cases}$$

The indexed frontier computes band placement directly from the priority and caches band extrema. It helps road-like cases but is not a universal improvement. Random and irregular graphs need split-capable behaviour, so the hybrid avoids applying the indexed structure where it is a poor fit.

7 Empirical Evidence

7.1 Correctness Tests

The test suite validates equality with Dijkstra on hand-built graphs, generated road-like grids, clustered graphs, random graphs, irregular weighted graphs, unreachable vertices, and adaptive variants v3 through v6. The core tested condition is:

$$d_{\mathrm{RFR}}(v) = d_{\mathrm{Dijkstra}}(v)$$

for all reachable vertices in each test instance.

7.2 Benchmark Smoke Results

Early smoke benchmarks showed exactness and useful work decomposition. Table 1 summarises representative v4 results from generated graph families.

Family	Correct	Global order avoided	Safe batches	Band splits
Road-like grid	true	754	37	1
Clustered basins	true	256	17	0
Random dense-ish	true	584	22	16
Irregular weighted	true	498	12	10

Table 1: Representative feedback-driven RFR smoke results. Structured cases allow many safe batches; ambiguous cases require more splitting.

7.3 Operational Graph Results

Operational benchmarking compares Python wall-clock time against Dijkstra on pre-built graph families. The current Python implementation is slower than Dijkstra in all tested graph-family runs. Table 2 summarises median v6 hybrid observations.

The interpretation is deliberately conservative. RFR avoids global ordering operations and exposes frontier ambiguity, but Python-level frontier machinery, safe-band scans, and local sorting dominate runtime for the current implementation.

Family	Time ratio	Efficiency	Safe batches
Road-like grid	4.85	1.89	37
Clustered basins	7.90	1.77	17
Random dense-ish	9.09	3.91	24
Irregular weighted	8.80	3.61	12

Table 2: Median operational results for the v6 hybrid frontier. All listed families were correct against DIJKSTRA. Exactness and work-shape improvements are validated; wall-clock superiority is not validated in this Python implementation.

7.4 Topographic Routing Case Study

The TopographicMaps simulator applies terrain Dijkstra and terrain RFR to Digital Elevation Model rasters. It compares target discovery work rather than full-map post-target exploration. In the current eight-sample public DEM set, the median target-work reduction is approximately 49.19%, with observed savings between 48.57% and 49.58%.

DEM	Dijkstra target work	RFR target work	Work saved
o41078a5	2386	1203	49.6%
o41078a1	2525	1286	49.1%
o41078a2	2475	1273	48.6%
o41078a3	2453	1249	49.1%
o41078a4	2367	1200	49.3%
o41078a6	2406	1213	49.6%
o41078a7	2424	1240	48.8%
i30dem	2048	1035	49.5%

Table 3: Topographic target-discovery work in the simulator. The metric compares global heap-order work for terrain DIJKSTRA against local band-resolution work for terrain RFR.

This case study supports the visual and diagnostic value of the method. It should not be read as a universal timing result because the simulator compares work units and because implementation details affect wall-clock performance.

8 Operational Applications and Implications

RFR is not only an alternative implementation of shortest paths. Its residual frontier model creates practical leverage in domains where the map is structured, where the frontier is interpretable, or where costs change over time. The most important applications are those where the graph is implicit or expensive to materialise, and where a solver must be integrated into a larger operational system rather than run as an isolated benchmark.

8.1 Direct Operation on Maps, Grids, and Rasters

Many real pathing domains begin as maps rather than abstract graphs: game tiles, navmesh regions, occupancy grids, terrain rasters, heatmaps, and GIS layers. A graph can be materialised from these structures, but doing so introduces construction cost, memory overhead, and another representation

boundary. The spatial components of this project show the practical value of treating each point as a local evaluator:

$$value(p) = \min_{q \in N(p)} value(q) + \Delta(q, p, \text{layers}).$$

In that setting, RFR's band and residual model can operate over structured frontier regions rather than over an arbitrary heap of graph vertices. This does not remove the need for a correctness-preserving ordering rule, but it aligns the ordering mechanism with the native structure of maps and rasters.

8.2 Games and Structured Level Pathfinding

Games commonly exploit structure through grids, navmeshes, hierarchical abstraction, and clustered regions. RFR is complementary to these practices. On a structured level, broad coherent bands may correspond to corridors, basins, rooms, road-like lanes, or terrain-cost contours. Safe-batch ratios, split pressure, and fallback rates can expose whether the level geometry is producing coherent pathing frontiers or chaotic ordering pressure.

That diagnostic value is useful during development. A designer or tools engineer can ask not only "what path was selected?" but also "where did the frontier become ambiguous?" and "which regions forced exact local ordering?" This makes RFR a potential development instrument even where another production solver remains the runtime default.

8.3 Dynamic Repathing

RFR is not yet an incremental shortest-path algorithm in the sense of D* Lite. However, its architecture points naturally toward local repair. Dynamic map changes usually affect a region of the frontier, not the entire metric space. Because RFR already represents the frontier as bands with residual pressure, updates could invalidate or refine only the affected bands, then resume propagation from the disturbed region.

This is a future-work direction rather than a validated claim. The important architectural point is that RFR separates the frontier into inspectable, transformable units. That is more amenable to local repair than an opaque global heap whose ordering state is meaningful only as a single total order.

8.4 Topographic and Terrain-Aware Routing

Terrain routing for drones, outdoor robotics, and GIS-style planning often combines distance, slope, resistance, masks, risk, and domain-specific movement costs. These costs tend to be spatially coherent. The TopographicMaps simulator demonstrates that RFR can expose and exploit this coherence under a target-query work metric while preserving the same terrain-cost answer as Dijkstra.

The operational implication is not that the current Python implementation is a production robotics solver. It is that terrain-aware routing is a natural fit for partial ordering: large regions of the frontier may be safely accepted together, while steep slopes, barriers, and high-resistance zones create visible residual pressure.

9 Relationship to Existing Techniques

Dijkstra enforces exactness by total global ordering. Delta-stepping groups tentative distances into buckets and can process light-edge relaxations in parallel, often trading strict ordering for controlled bucket processing. RFR is related in spirit but differs in two important ways:

1. RFR treats bucket or band processing as a safety problem, not only as a fixed-width scheduling choice.

2. RFR records residual pressure and can split or locally resolve an unsafe band rather than relying solely on a static bucket width.

The practical comparison target is therefore not only DIJKSTRA, but also bucketed, radix, delta-stepping, and domain-specialised shortest-path implementations.

9.1 Incremental Search

Incremental methods such as D* Lite reuse previous search information when edge costs change. RFR does not currently implement D* Lite's incremental consistency machinery. Its relevance is architectural: residual bands identify where ordering uncertainty lives, which could provide a natural repair unit for future dynamic variants. A dynamic RFR extension would need to define invalidation, band repair, and predecessor repair rules explicitly before claiming equivalence to established incremental algorithms.

9.2 Hierarchical Pathfinding

Hierarchical methods such as HPA* reduce search cost by abstracting a map into clusters and solving at multiple resolutions. RFR addresses a different layer of the problem. It does not replace hierarchy; it changes how a frontier is ordered inside a level of representation. The two approaches are compatible: hierarchy can reduce the search domain, while RFR can resolve the frontier within a cluster, corridor, or refined local region.

This distinction is important for games. A production game pathing stack may still use navmeshes, HPA*, flow fields, or cached tactical maps. RFR's contribution is a safety-checked partial ordering mechanism that can sit inside such systems, especially where residual diagnostics are valuable.

10 Limitations and Non-Claims

The current state of RFR supports several claims and explicitly rejects others.

10.1 Validated Claims

- RFR preserves exact shortest-path distances on tested non-negative graph families.
- Structured frontiers can be processed in safe batches.
- The algorithm exposes where ordering pressure occurs through residual history, splits, fallback, and safe-batch ratios.
- Adaptive policy revision improves behaviour when static graph features misclassify frontier ambiguity.
- The topographic simulator shows a clear target-work reduction under the current work metric.
- The architecture creates practical integration points for map-native routing, diagnostics, and future local repair.

10.2 Non-Claims

- The current Python graph implementation is not faster than DIJKSTRA on pre-built graph benchmarks.
- RFR is not expected to outperform DIJKSTRA on every graph family.
- Resolution efficiency is not identical to wall-clock speed.
- The indexed frontier is not universally beneficial; it is useful only under policies that predict coherent structure.
- The present implementation is not yet an incremental replanning algorithm and should not be represented as a replacement for D* Lite.
- The present work does not claim to solve multi-agent pathfinding or conflict resolution.

10.3 Implementation Bottlenecks

Current bottlenecks include:

- Python-level band management overhead,
- safe-band scans,
- local sorting and local heap fallback,
- insufficiently specialised data structures for ambiguous frontiers,
- lack of comparison against bucketed and radix shortest-path baselines,
- absence of validated incremental repair machinery for dynamic maps.

11 Future Work

Promising next steps are:

1. implement a low-overhead C++ or Rust frontier core to test wall-clock performance without Python interpreter overhead,
2. implement a split-capable indexed frontier,
3. add a cluster-specific basin frontier,
4. reduce safe-band sorting in coherent road-like frontiers,
5. compare against delta-stepping, radix heaps, and specialised integer-weight solvers,
6. run larger benchmarks where global ordering cost may dominate interpreter overhead,
7. distinguish target-query work from full-field computation in all demos and reports,
8. develop an incremental RFR variant for dynamic costs and local repair,

9. integrate RFR within hierarchical game pathing stacks such as clustered grids or navmesh regions,

10. treat direct raster and grid operation as a first-class benchmark target rather than a secondary visual demo.

12 Conclusion

Residual Frontier Refinement reframes exact shortest-path propagation as a problem of resolving only the frontier ordering uncertainty that can affect correctness. It replaces unconditional global heap ordering with conservative band safety checks, residual-guided splitting, and local exact fallback. The current implementation validates exactness, provides a useful diagnostic account of frontier ambiguity, and shows clear work-shape benefits on structured and spatial cases. It does not yet validate universal wall-clock superiority over DIJKSTRA in Python. The most defensible conclusion is that global total ordering is not inherent to shortest-path correctness; it is one implementation strategy, and RFR demonstrates an exact alternative based on adaptive residual partial ordering.

References

[1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[2] U. Meyer and P. Sanders. Delta-stepping: A parallelizable shortest path algorithm. *Journal of Algorithms*, 49(1):114–152, 2003.

[3] S. Koenig and M. Likhachev. D* Lite. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2002.

[4] A. Botea, M. Müller, and J. Schaeffer. Near optimal hierarchical path-finding. *Journal of Game Development*, 1(1):7–28, 2004.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, third edition, 2009.